Anatomía de un LLM

Fundamentos matemáticos de los modelos de lenguaje

Fase VI · Diseño y construcción de optimizadores con IA

Módulo 25

Modelos sustitutos, diseño de experimentos y optimización bayesiana

Cómo optimizar procesos caros y ruidosos con pocos experimentos

---

Resumen del módulo

Edición de octubre de 2026

Sobre este resumen

Este documento reúne lo indispensable de los apuntes del Módulo 25: el texto expositivo, todas las definiciones y todos los resultados (teoremas, proposiciones, lemas y corolarios), las tablas y las observaciones de contenido. Se han suprimido las demostraciones, los ejemplos, las lecturas, las figuras, los ejercicios y sus soluciones. La numeración de temas, secciones, enunciados y ecuaciones es la misma que en los apuntes, de modo que cualquier referencia puede buscarse allí; las que remiten a elementos suprimidos (un ejemplo, una figura, una ecuación de una demostración) conservan su número en los apuntes.

Notación

Símbolo	Significado
$x \in \mathcal{X} \subset \mathbb{R}^d$	vector de variables de decisión (parámetros del proceso) en el dominio $\mathcal{X}$
$f(x)$, $y = f(x) + \varepsilon$	función objetivo (a minimizar) y observación con ruido $\varepsilon \sim \mathcal{N}(0, \sigma_n^2)$
$c_j(x) \leq 0$	restricciones
$\mathcal{D}_n = \{(x_i, y_i)\}_{i=1}^n$	datos tras n evaluaciones; N es el presupuesto total
$u \in [-1, 1]^d$	variables codificadas de un diseño de experimentos
$k(x, x')$, K, $K_y = K + \sigma_n^2 I$	núcleo (covarianza), matriz de Gram y matriz con ruido
ℓ, σ_f^2, θ	longitud de escala, varianza de la señal y vector de hiperparámetros
$\mu_n(x)$, $\sigma_n(x)$	media y desviación típica posteriores de $f(x)$ dados $\mathcal{D}_n$
f_n^*	mejor valor (mínimo) de referencia tras n evaluaciones
φ, Φ	densidad y función de distribución de la normal estándar
$\alpha_n(x)$	función de adquisición; EI, PI, LCB sus casos principales
$r_N = f(\hat{x}_N) - f(x^\star)$	regret simple de la recomendación $\hat{x}_N$
λ, Λ	configuración de hiperparámetros y espacio de configuraciones

Índice

Sobre este resumen **ii**

1 Optimizar procesos caros y ruidosos **1**

1.1 El problema de la caja negra . . . 1
1.2 Presupuesto y ruido . . . 2
1.3 Restricciones . . . 2
1.4 El bucle de optimización con un modelo sustituto . . . 3
1.5 Caso de estudio: ajuste de una inyectora . . . 3

2 Diseño de experimentos **5**

2.1 Principios y variables codificadas . . . 5
2.2 El diseño factorial 2^k . . . 5
2.3 Análisis de la varianza . . . 6
2.4 Factoriales fraccionados . . . 6
2.5 Superficies de respuesta . . . 7
2.6 Diseños para experimentos de ordenador: el hipercubo latino . . . 7
2.7 Caso de estudio: cribado de la inyectora con un 2^3 . . . 8

3 Modelos sustitutos **9**

3.1 Qué se pide a un sustituto . . . 9
3.2 Regresión polinómica y de bases . . . 9
3.3 Procesos gaussianos . . . 9
3.4 Hiperparámetros por verosimilitud marginal . . . 10
3.5 Redes neuronales como sustitutos . . . 11
3.6 Redes informadas por la física . . . 11
3.7 Caso de estudio: tres sustitutos del coste . . . 12

4 Optimización bayesiana **13**

4.1 El algoritmo . . . 13
4.2 Probabilidad de mejora . . . 13
4.3 Mejora esperada . . . 14
4.4 Cotas de confianza y muestreo de Thompson . . . 14
4.5 Optimizar la función de adquisición . . . 14
4.6 Restricciones . . . 15

4.7 Evaluaciones por lotes . . . 15
4.8 Varios objetivos . . . 15
4.9 Varias fidelidades . . . 16
4.10 Caso de estudio: el optimizador de la inyectora . . . 16

5 Casos de estudio: moldeo por inyección y hormigón 18
5.1 Del problema al optimizador . . . 18
5.2 Caso A: el ajuste completo de la inyectora . . . 19
5.3 Caso B: un hormigón con menos CO_2 . . . 19

6 Ajuste de hiperparámetros, validación y parada 20
6.1 El ajuste de hiperparámetros como optimización de caja negra . . . 20
6.2 Búsqueda aleatoria frente a rejilla . . . 20
6.3 El estimador de Parzen con estructura de árbol (TPE) . . . 21
6.4 Evaluaciones parciales: Successive Halving y Hyperband . . . 21
6.5 Validación del sustituto . . . 22
6.6 Cuándo dejar de experimentar . . . 22
6.7 Caso de estudio: ajuste de un *gradient boosting* . . . 23

Bibliografía 24

Tema 1

Optimizar procesos caros y ruidosos

Este tema plantea el problema común a todo el módulo: minimizar una función que solo se conoce a través de experimentos caros y ruidosos, con un presupuesto fijo de evaluaciones y con restricciones que a veces tampoco se conocen de antemano. La formalización sigue el capítulo 1 de Garnett [19] y la sección 2 del tutorial de Frazier [15]; la discusión sobre cuándo compensa un modelo sustituto sigue la §10.1 de Martins y Ning [31], y los planes de muestreo en rejilla, la §13.1 de Kochenderfer y Wheeler [26].

1.1 El problema de la caja negra

En el entrenamiento de una red (módulo 7) la función objetivo es una fórmula explícita, su gradiente se obtiene por diferenciación automática (módulo 10) y evaluarla en un minilote cuesta milisegundos. En un proceso industrial nada de esto se cumple: para conocer la tasa de defectos de una inyectora con un ajuste dado hay que fabricar un lote de piezas e inspeccionarlo.

Definición 1.1 (Problema de optimización de caja negra). Sean $\mathcal{X} \subset \mathbb{R}^d$ un dominio compacto (habitualmente un hiperrectángulo $\prod_j [a_j, b_j]$), $f\colon \mathcal{X} \to \mathbb{R}$ una función desconocida y $N \in \mathbb{N}$ un *presupuesto* de evaluaciones. Un *oráculo* devuelve, para cada $x \in \mathcal{X}$ elegido, una observación

$$y = f(x) + \varepsilon, \qquad \mathbb{E}[\varepsilon] = 0, \quad \operatorname{Var}(\varepsilon) = \sigma_n^2(x),$$

con ruidos independientes entre evaluaciones. Una *política* elige secuencialmente x_{n+1} a partir de los datos $\mathcal{D}_n = \{(x_i, y_i)\}_{i \le n}$ y, agotado el presupuesto, emite una *recomendación* $\hat{x}_N \in \mathcal{X}$. El objetivo es que $f(\hat{x}_N)$ sea lo más próximo posible a $\min_{x \in \mathcal{X}} f(x)$.

Tres rasgos distinguen este problema de los de los módulos 7 y 8: no hay gradiente (ni siquiera ruidoso), el número de evaluaciones es pequeño (de diez a unos centenares) y el coste de decidir dónde evaluar es despreciable frente al de evaluar. Esta última asimetría justifica gastar mucho cómputo en cada decisión: ajustar un modelo estadístico de f y optimizar sobre él [19, §1.1].

Definición 1.2 (Regret). Sea $x^\star \in \arg\min_{\mathcal{X}} f$. El *regret simple* de la recomendación es $r_N = f(\hat{x}_N) - f(x^\star) \ge 0$. El *regret acumulado* de las evaluaciones es $R_N = \sum_{n=1}^{N} \big(f(x_n) - f(x^\star)\big)$.

La elección entre ambos depende del caso. En un estudio de laboratorio solo importa la receta final: se mide r_N. Si los experimentos se hacen en la línea de producción y las piezas ensayadas se venden o se desechan, cada experimento malo cuesta dinero y se mide R_N; Garnett [19, §6.1–6.2] formaliza ambas utilidades.

Observación 1.3. La recomendación $\hat{x}_N$ no tiene por qué ser uno de los puntos evaluados ni el de mejor observación. Con ruido, el mínimo observado es un mal estimador del mínimo

verdadero (proposición 1.6); en la práctica se recomienda el punto que minimiza un modelo ajustado a todos los datos y se confirma con ensayos repetidos.

1.2 Presupuesto y ruido

Proposición 1.4 (Maldición de la dimensión para rejillas). *Sea $\mathcal{X} = [0,1]^d$ y sea $h \in (0, \frac{1}{2}]$. Si un conjunto de N puntos garantiza que todo $x \in \mathcal{X}$ dista menos de h en norma del máximo de algún punto del conjunto, entonces $N \geq (2h)^{-d}$. Una rejilla de m niveles por variable tiene m^d puntos.*

Con $h = 0{,}1$ (un décimo del rango de cada variable) hacen falta al menos 5^d puntos: 125 con tres variables y casi diez millones con diez. Ningún proceso físico admite ese presupuesto; de ahí la necesidad de diseños que no crezcan exponencialmente (tema 2) y de modelos que interpolen entre ensayos (tema 3).

El ruido tiene dos orígenes: la variabilidad del proceso (lote de materia prima, temperatura ambiente, desgaste) y el error de medida. Cuando la respuesta es una proporción estimada con m piezas, el ruido es binomial.

Proposición 1.5. *Si en un lote de m piezas cada una es defectuosa con probabilidad d, independientemente, la fracción observada $\hat{d}$ cumple $\mathbb{E}[\hat{d}] = d$ y $\mathrm{Var}(\hat{d}) = d(1-d)/m$.*

La consecuencia más traicionera del ruido es que el mejor resultado observado es sistemáticamente optimista.

Proposición 1.6 (Maldición del ganador). *Sean $y_i = f(x_i) + \varepsilon_i$, $i = 1, \dots, n$, con ε_i independientes de media nula. Entonces*

$$\mathbb{E}\Big[\min_i y_i\Big] \leq \min_i f(x_i).$$

Si además todos los $f(x_i)$ son iguales a f_0 y $\varepsilon_i \sim \mathcal{N}(0, \sigma^2)$, entonces $\mathbb{E}[\min_i y_i] = f_0 - \sigma\, e_n$, con $e_n = \mathbb{E}[\max_i Z_i]$ para $Z_i \sim \mathcal{N}(0,1)$ independientes.

Los valores de e_n se obtienen por integración numérica: $e_2 = 1/\sqrt{\pi} \approx 0{,}5642$ (ejercicio 1.4), $e_5 \approx 1{,}163$, $e_{10} \approx 1{,}539$ y $e_{30} \approx 2{,}043$. Elegir el mejor de diez ensayos igual de buenos produce un resultado aparentemente mejor en más de 1,5 desviaciones típicas.

1.3 Restricciones

Las restricciones de un proceso son de naturaleza muy distinta, y conviene clasificarlas antes de elegir el método [19, §11.2], [26, §1.4]:

- *conocidas y baratas*: los límites de la máquina, la suma de las fracciones de una mezcla igual a 1, una relación entre variables. Se imponen directamente al generar candidatos;
- *desconocidas y caras*: una especificación de calidad que solo se comprueba con el experimento (alabeo máximo, resistencia mínima). Se modelan igual que el objetivo y se trata su probabilidad de cumplirse (tema 4);
- *ocultas*: zonas donde el experimento falla y no produce dato (la pieza sale incompleta, la simulación no converge). Se modelan como una clasificación;

- *de seguridad*: no basta con acabar en un punto factible, sino que no se debe *ensayar* un punto peligroso (presión excesiva, temperatura de degradación del polímero). Exigen métodos de exploración segura [26, §16.6].

Definición 1.7 (Problema con restricciones de caja negra). Se busca $\min_{x \in \mathcal{X}} f(x)$ sujeto a $c_j(x) \leq 0$, $j = 1, \ldots, J$, donde cada c_j es una función desconocida que se observa con ruido en el mismo experimento que f o en uno distinto. El regret simple se define respecto al mejor punto *factible*, y una recomendación no factible se considera un fracaso, sea cual sea su valor de f.

1.4 El bucle de optimización con un modelo sustituto

Todos los métodos del módulo comparten la estructura de la figura 1.1. Un diseño inicial (tema 2) proporciona los primeros datos; un modelo sustituto (tema 3) resume lo aprendido con su incertidumbre; una función de adquisición (tema 4) traduce el modelo en la decisión del siguiente experimento; y una regla de parada (tema 6) decide cuándo dejar de experimentar.

Familia	Cuándo es adecuada	Limitaciones
Rejilla y un factor cada vez	Una o dos variables, evaluaciones baratas	Crece como m^d; no detecta interacciones
Diseño de experimentos y superficie de respuesta	2–8 variables, decenas de ensayos, se quiere entender el proceso	Modelo polinómico local
Optimización bayesiana	1–20 variables, 10–500 evaluaciones caras y ruidosas	Coste cúbico en n; peor en dimensión alta
Evolutivos (CMA-ES, genéticos)	Miles de evaluaciones baratas, paisajes irregulares	Poco eficientes en número de evaluaciones
Gradiente (adjunto, autodiferenciación)	Simuladores diferenciables, muchas variables	Requiere acceso al código del modelo

Cuadro 1.1: Familias de métodos según el presupuesto y la dimensión. El límite de unas veinte variables para la optimización bayesiana procede de Frazier [15, §1].

La §10.1 de Martins y Ning [31] resume el criterio económico: un sustituto compensa cuando el coste de construirlo y consultarlo es pequeño frente al de las evaluaciones que ahorra, cuando el ruido impide usar diferencias finitas y cuando se quiere reutilizar el modelo para otras preguntas (sensibilidad, robustez).

1.5 Caso de estudio: ajuste de una inyectora

Herramienta	Para qué sirve	Licencia	Cuándo elegirla
Ax [4]	Plataforma de experimentación adaptativa: bucle completo, restricciones, varios objetivos, lotes	MIT	Experimentos reales gestionados por un equipo
BoTorch [5]	Optimización bayesiana sobre PyTorch con adquisiciones de Monte Carlo	MIT	Investigación o adquisiciones a medida
Optuna [34]	Ajuste de hiperparámetros con TPE, GP y poda	MIT	Ajuste de modelos de aprendizaje automático
SMAC3 [29]	Optimización bayesiana con bosques aleatorios	BSD	Espacios mixtos y condicionales
SciPy [39]	`optimize` y `stats.qmc`: métodos clásicos y planes de muestreo	BSD	Líneas base y diseños

Cuadro 1.2: Tecnologías para el bucle completo de optimización de caja negra.

Tema 2

Diseño de experimentos

Antes de que existiera la optimización bayesiana, la estadística industrial ya había resuelto una parte esencial del problema: decidir qué ensayos hacer *antes* de ver ningún resultado para extraer el máximo de información con el mínimo de experimentos. Este tema sigue los capítulos 5, 6, 8 y 11 de Montgomery [33], texto de referencia de los cursos de diseño de experimentos, y las lecciones 6 y 11 del curso STAT 503 de Penn State [35]. Las superficies de respuesta proceden de Box y Wilson [8]; el hipercubo latino, de McKay, Beckman y Conover [32], y los diseños para experimentos de ordenador, del capítulo 13 de Kochenderfer y Wheeler [26].

2.1 Principios y variables codificadas

Montgomery [33, §1.4] resume el diseño de experimentos en tres principios. La *aleatorización* del orden de los ensayos protege contra factores no controlados que derivan en el tiempo (calentamiento de la máquina, humedad); la *replicación* permite estimar el error puro y promediarlo; el *bloqueo* agrupa los ensayos en condiciones homogéneas (un mismo lote de materia prima, un mismo turno) para eliminar esa fuente de variabilidad de la comparación.

Definición 2.1 (Variables codificadas). Si la variable física x_j se ensaya en el intervalo $[a_j, b_j]$, su versión codificada es

$$u_j = \frac{x_j - c_j}{h_j}, \qquad c_j = \frac{a_j + b_j}{2}, \quad h_j = \frac{b_j - a_j}{2},$$

de modo que el nivel bajo corresponde a $u_j = -1$, el alto a $u_j = +1$ y el centro a $u_j = 0$.

Con variables codificadas todos los coeficientes de un modelo tienen las mismas unidades que la respuesta y son comparables entre sí; además, las columnas de los diseños factoriales resultan ortogonales [33, §6.9].

2.2 El diseño factorial 2^k

Definición 2.2 (Factorial 2^k y efectos). Un diseño factorial 2^k ensaya las $N = 2^k$ combinaciones de k factores a dos niveles $u_j \in \{-1, +1\}$, cada una con n réplicas. Su *matriz de diseño* $U \in \{-1, 1\}^{N \times k}$ se escribe en *orden estándar* (la primera columna alterna en cada fila, la segunda cada dos, la j-ésima cada 2^{j-1}). La columna de la interacción entre los factores de un conjunto S es el producto elemento a elemento $\prod_{j \in S} U_{\cdot j}$. El *efecto* del término asociado a una columna $m \in \{-1, 1\}^N$ es

$$\hat{E} = \bar{y}_+ - \bar{y}_- = \frac{2}{N} \sum_{i=1}^{N} m_i \, \bar{y}_i,$$

donde $\bar{y}_i$ es la media de las réplicas del ensayo i y $\bar{y}_\pm$ son las medias de los ensayos con $m_i = \pm 1$. La cantidad $\sum_i m_i\,(n\bar{y}_i)$ se llama *contraste*.

Proposición 2.3 (Ortogonalidad). *Sea $M \in \{-1,1\}^{N\times(N-1)}$ la matriz con las columnas de todos los efectos principales e interacciones de un 2^k, y $X = [\mathbf{1}\ M]$. Entonces $X^\top X = N\,I_N$.*

Corolario 2.4. *El ajuste por mínimos cuadrados del modelo $y = \beta_0 + \sum_S \beta_S \prod_{j\in S} u_j + \varepsilon$ a las medias $\bar{y}_i$ tiene solución $\hat{\beta}_S = \frac{1}{N}\sum_i m_{iS}\bar{y}_i = \hat{E}_S/2$: cada coeficiente es la mitad del efecto, y no depende de qué otros términos se incluyan en el modelo.*

Teorema 2.5 (Optimalidad del 2^k). *Sea un diseño cualquiera de N ensayos en $[-1,1]^k$ para el modelo de primer orden $y = \beta_0 + \sum_j \beta_j u_j + \varepsilon$, con errores independientes de varianza σ^2 y matriz X de rango completo. Entonces $\mathrm{Var}(\hat{\beta}_j) \geq \sigma^2/N$ para todo j, y el factorial 2^k (o cualquier fracción ortogonal de él con N ensayos) alcanza la cota.*

Montgomery [33, §6.7] enuncia este resultado diciendo que los 2^k son diseños óptimos: con el mismo número de ensayos, ningún otro estima los efectos con menor varianza.

2.3 Análisis de la varianza

El análisis de la varianza (ANOVA) decide qué efectos son distinguibles del ruido. La teoría de contrastes es la del módulo 0.9, §5; aquí se necesita la versión para efectos de un factorial.

Proposición 2.6 (Descomposición de la variabilidad). *En un 2^k con n réplicas, sea $SS_T = \sum_{i,r}(y_{ir} - \bar{y})^2$ la suma de cuadrados total. Entonces*

$$SS_T = \sum_{S\neq\varnothing} SS_S + SS_E, \qquad SS_S = \frac{nN}{4}\hat{E}_S^2, \qquad SS_E = \sum_{i,r}(y_{ir} - \bar{y}_i)^2,$$

con $N-1$ grados de libertad para los efectos (uno cada uno) y $N(n-1)$ para el error. Si los errores son normales de varianza σ^2 y $E_S = 0$, el cociente $F_S = SS_S/MS_E$, con $MS_E = SS_E/(N(n-1))$, sigue una distribución $F(1, N(n-1))$.

2.4 Factoriales fraccionados

Con $k = 7$ factores, el factorial completo tiene 128 ensayos, y la mayoría de sus grados de libertad estiman interacciones de tres o más factores, casi siempre despreciables. Los factoriales fraccionados ensayan solo una fracción 2^{-p} del total [33, cap. 8].

Definición 2.7 (Fracción 2^{k-p}, alias y resolución). Una fracción 2^{k-p} se construye con un factorial completo en $k-p$ factores básicos y p *generadores* que definen los demás como productos de los básicos (por ejemplo $D = ABC$). Cada generador da una palabra de la *relación definidora* ($I = ABCD$); el conjunto de todas sus palabras y productos es el subgrupo definidor. Dos efectos son *alias* si sus columnas coinciden en la fracción: el alias de un efecto se obtiene multiplicándolo por cada palabra y simplificando con $A^2 = I$. La *resolución* es la longitud de la palabra más corta de la relación definidora.

En resolución III los efectos principales tienen alias en interacciones dobles; en resolución IV, los principales están libres de dobles pero las dobles tienen alias entre sí; en resolución

V, principales y dobles están libres entre sí [33, §8.6–8.7]. La elección habitual en una fase de cribado es resolución IV.

2.5 Superficies de respuesta

Los factoriales a dos niveles estiman efectos lineales e interacciones, pero no curvatura: no pueden localizar un óptimo interior. La metodología de superficies de respuesta de Box y Wilson [8] procede por etapas: un modelo de primer orden para avanzar por la dirección de máximo descenso hasta la región del óptimo y, allí, un modelo de segundo orden [33, §11.1–11.3], [35, lección 11].

Definición 2.9 (Modelo de segundo orden y diseño central compuesto). El modelo de segundo orden en k variables codificadas es

$$\hat{y}(u) = b_0 + b^\top u + u^\top B u, \qquad B = B^\top, \quad B_{jj} = b_{jj},\ B_{jl} = \tfrac{1}{2} b_{jl}\ (j \neq l),$$

con $1 + k + k(k+1)/2$ coeficientes. El *diseño central compuesto* (CCD) combina un 2^k (o una fracción de resolución V), $2k$ puntos axiales $(\pm\alpha, 0, \dots, 0), \dots$ y n_c puntos centrales. Es *rotable* (la varianza de la predicción solo depende de $\|u\|$) si $\alpha = (2^k)^{1/4}$.

Proposición 2.10 (Punto estacionario y análisis canónico). *Si B es invertible, el único punto crítico de $\hat{y}$ es $u_s = -\frac{1}{2} B^{-1} b$, con valor $\hat{y}(u_s) = b_0 + \frac{1}{2} b^\top u_s$. Si $B = Q \Lambda Q^\top$ es su descomposición espectral y $w = Q^\top (u - u_s)$, entonces $\hat{y}(u) = \hat{y}(u_s) + \sum_j \lambda_j w_j^2$. El punto es un mínimo si todos los $\lambda_j > 0$, un máximo si todos son negativos y un punto de silla en otro caso.*

2.6 Diseños para experimentos de ordenador: el hipercubo latino

Cuando la respuesta procede de una simulación determinista no tiene sentido replicar, y cuando se va a ajustar un modelo flexible (tema 3) interesa cubrir el dominio uniformemente en lugar de concentrar los ensayos en los vértices. Montgomery [33, §11.5] y Kochenderfer y Wheeler [26, §13.3–13.5] tratan estos diseños de relleno del espacio.

Definición 2.12 (Hipercubo latino). Un *hipercubo latino* (LHS) de n puntos en $[0,1)^d$ es un conjunto $\{z_1, \dots, z_n\}$ tal que, para cada coordenada j, cada estrato $[k/n, (k+1)/n)$, $k = 0, \dots, n-1$, contiene exactamente una de las z_{ij}. Se construye con d permutaciones independientes π_j de $\{0, \dots, n-1\}$ y uniformes $v_{ij} \in [0,1)$: $z_{ij} = (\pi_j(i) + v_{ij})/n$.

Toda proyección de un LHS sobre una coordenada está perfectamente repartida: si al final solo una variable resulta relevante, los n ensayos dan n niveles distintos de ella, mientras que una rejilla de n puntos en d dimensiones da solo $n^{1/d}$. Este argumento reaparece en el tema 6 para la búsqueda aleatoria.

Teorema 2.13 (Reducción de varianza en una dimensión). *Sea $g\colon [0,1) \to \mathbb{R}$ de cuadrado integrable, $\mu = \int_0^1 g$, y sean $\hat{\mu}_{MC} = \frac{1}{n}\sum_i g(V_i)$ con V_i uniformes independientes y $\hat{\mu}_E = \frac{1}{n}\sum_k g\big((k + V_k)/n\big)$ el estimador estratificado (un LHS en $d = 1$). Ambos son insesgados y $\operatorname{Var}(\hat{\mu}_E) \leq \operatorname{Var}(\hat{\mu}_{MC})$.*

En dimensión d, McKay, Beckman y Conover [32, §2.1] demostraron que el LHS reduce la varianza del estimador de la media respecto al muestreo aleatorio siempre que la respuesta sea monótona en cada variable. En la práctica, al estimar el coste medio de la inyectora en todo el dominio con 20 puntos, la desviación típica del estimador baja de 0,0247 (Monte Carlo) a 0,0129 (LHS): la varianza se reduce a la cuarta parte.

Definición 2.14 (Criterio maximín). Entre varios diseños de n puntos se prefiere el que maximiza la distancia mínima entre pares, $\min_{i<l}\|z_i - z_l\|$, o, equivalentemente en espíritu, el que minimiza una *discrepancia* (diferencia entre la fracción de puntos en cada caja y su volumen).

Un LHS aleatorio puede ser malo (en el peor caso, todos los puntos en la diagonal); por eso se optimiza. En la práctica, con 10 puntos en tres dimensiones, la distancia mínima media pasa de 0,168 con puntos uniformes a 0,228 con un LHS, y la optimización `random-cd` de SciPy reduce la discrepancia centrada de 0,0154 a 0,0105.

2.7 Caso de estudio: cribado de la inyectora con un 2^3

Herramienta	Para qué sirve	Licencia	Cuándo elegirla
pyDOE3 [36]	Factoriales completos y fraccionados, Plackett–Burman, CCD, Box–Behnken, LHS, diseños óptimos	BSD-3	Generar diseños clásicos en Python
`scipy.stats.qmc` [39]	LHS (con optimización `random-cd` y `lloyd`), Sobol, Halton, discrepancias	BSD	Diseños de relleno del espacio
statsmodels	ANOVA y regresión con tablas de contraste	BSD-3	Análisis estadístico completo
R (FrF2, rsm)	Fraccionados con tablas de alias; superficies de respuesta	GPL	Estudios estadísticos detallados
Minitab, JMP	Diseño y análisis con interfaz gráfica	Comerciales	Entornos industriales con formación previa

Cuadro 2.1: Tecnologías para el diseño y el análisis de experimentos.

Tema 3

Modelos sustitutos

Un modelo sustituto (*surrogate*) es una aproximación barata de la función desconocida, ajustada a los experimentos ya hechos. Para optimizar no basta con que prediga bien: debe decir también *cuánto* se fía de cada predicción, porque esa incertidumbre es la que permite decidir dónde merece la pena experimentar. Este tema estudia tres familias: la regresión polinómica y de bases (§14.2–14.5 de Kochenderfer y Wheeler [26]), los procesos gaussianos (capítulos 2, 4 y 5 de Rasmussen y Williams [38] y capítulos 2–4 de Garnett [19]) y las redes neuronales, con los *ensembles* de Lakshminarayanan *et al.* [27] y las redes informadas por la física de Raissi *et al.* [37].

3.1 Qué se pide a un sustituto

Un sustituto útil para optimizar debe (i) interpolar o suavizar los datos según haya o no ruido, (ii) dar una incertidumbre calibrada, mayor lejos de los datos, (iii) ajustarse en segundos con decenas o cientos de puntos y (iv) admitir conocimiento previo (suavidad, monotonía, física). Ningún modelo es el mejor en todo: la tabla 3.1 y el caso del final del tema comparan las opciones.

3.2 Regresión polinómica y de bases

Sea $\phi\colon \mathcal{X} \to \mathbb{R}^p$ un vector de funciones base (monomios hasta grado 2, por ejemplo) y $\Phi \in \mathbb{R}^{n\times p}$ la matriz con filas $\phi(x_i)^\top$. El ajuste por mínimos cuadrados es $\hat\beta = (\Phi^\top\Phi)^{-1}\Phi^\top y$ (módulo 8, §2.2) y las predicciones en los datos son $\hat y = Hy$, con la *matriz sombrero* $H = \Phi(\Phi^\top\Phi)^{-1}\Phi^\top$. Un polinomio completo de grado 2 en d variables tiene $p = (d+1)(d+2)/2$ coeficientes: 10 con $d=3$ y 66 con $d=10$, lo que limita su uso a dimensión baja.

Proposición 3.1 (Validación cruzada dejando uno fuera sin reajustar). *Sea $e_i = y_i - \hat y_i$ el residuo del ajuste con todos los datos y h_{ii} el i-ésimo elemento diagonal de H, con $h_{ii} < 1$. Entonces el residuo de predecir y_i con el modelo ajustado sin el punto i es*

$$e_{(i)} = \frac{e_i}{1 - h_{ii}}.$$

La raíz del error cuadrático medio de los $e_{(i)}$ (PRESS) es una estimación honesta del error de predicción con el coste de un solo ajuste [26, §14.5]; el tema 6 obtiene el resultado análogo para procesos gaussianos.

3.3 Procesos gaussianos

Definición 3.2 (Proceso gaussiano). Un *proceso gaussiano* con función media $m\colon \mathcal{X} \to \mathbb{R}$ y núcleo (función de covarianza) $k\colon \mathcal{X}\times\mathcal{X} \to \mathbb{R}$, $f \sim \mathcal{GP}(m, k)$, es una colección de

variables aleatorias $\{f(x)\}_{x\in\mathcal{X}}$ tal que para cualesquiera $x_1,\dots,x_n$ el vector $(f(x_1),\dots,f(x_n))$ es normal multivariante de media $(m(x_i))_i$ y covarianza $K_{il}=k(x_i,x_l)$ [38, cap. 2].

Para que la definición tenga sentido, toda matriz K así construida debe ser simétrica semidefinida positiva: k debe ser un *núcleo semidefinido positivo*. Las sumas y los productos de núcleos lo son, así como los productos por constantes positivas [38, cap. 4].

Definición 3.3 (Núcleos habituales). Sea $r=\|x-x'\|/\ell$ (o, con *relevancia automática* ARD, $r^2=\sum_j (x_j-x'_j)^2/\ell_j^2$). Se definen

$$k_{SE}(x,x')=\sigma_f^2\exp\!\big(-\tfrac12 r^2\big),\qquad k_{5/2}(x,x')=\sigma_f^2\Big(1+\sqrt{5}\,r+\tfrac{5}{3}r^2\Big)e^{-\sqrt{5}\,r}.$$

El primero (exponencial cuadrático) produce funciones infinitamente derivables; el segundo (Matérn con $\nu=5/2$), funciones dos veces derivables [38, cap. 4].

La longitud de escala ℓ mide la distancia a la que dos valores de f dejan de estar correlacionados; con ARD, una ℓ_j grande indica que la variable j apenas influye. Garnett [19, §3.3] y Frazier [15, §3.1] recomiendan Matérn 5/2 para optimización: el exponencial cuadrático supone una suavidad que los procesos físicos rara vez tienen.

Lema 3.4 (Condicionamiento de la normal). *Sea (a,b) un vector normal con $a\in\mathbb{R}^p$, $b\in\mathbb{R}^q$, medias μ_a,μ_b y covarianzas $\Sigma_{aa},\Sigma_{ab},\Sigma_{bb}$, con Σ_{bb} invertible. Entonces*

$$a\mid b\sim\mathcal{N}\big(\mu_a+\Sigma_{ab}\Sigma_{bb}^{-1}(b-\mu_b),\ \Sigma_{aa}-\Sigma_{ab}\Sigma_{bb}^{-1}\Sigma_{ba}\big).$$

Teorema 3.5 (Distribución posterior de un proceso gaussiano). *Sea $f\sim\mathcal{GP}(0,k)$ y $y_i=f(x_i)+\varepsilon_i$ con $\varepsilon_i\sim\mathcal{N}(0,\sigma_n^2)$ independientes. Sean $K_y=K+\sigma_n^2 I$ y $k_*(x)=(k(x,x_1),\dots,k(x,x_n))^\top$. Dado $\mathcal{D}_n$, $f(x)$ es normal con*

$$\mu_n(x)=k_*(x)^\top K_y^{-1}y,\qquad \sigma_n^2(x)=k(x,x)-k_*(x)^\top K_y^{-1}k_*(x).$$

Observación 3.6. (i) La media es una combinación lineal de los datos: $\mu_n(x)=\sum_i \alpha_i k(x,x_i)$ con $\alpha=K_y^{-1}y$. (ii) La varianza no depende de los valores y, solo de dónde se ha medido, y nunca supera la previa $k(x,x)$ (ejercicio 3.7). (iii) Para predecir una nueva *observación* $y(x)$, y no $f(x)$, se suma σ_n^2; confundir ambas varianzas es un error frecuente al calcular funciones de adquisición (tema 4). (iv) Con un núcleo lineal $k(x,x')=\sigma_f^2 x^\top x'$, el proceso gaussiano es la regresión lineal bayesiana, cuya media posterior es la de la regresión ridge (módulo 8, §6.2).

3.4 Hiperparámetros por verosimilitud marginal

Los hiperparámetros $\theta=(\sigma_f,\ell_1,\dots,\ell_d,\sigma_n)$ no se conocen. El criterio habitual es maximizar la probabilidad de los datos observados con f integrada [38, cap. 5].

Proposición 3.8 (Verosimilitud marginal). *Con $f\sim\mathcal{GP}(0,k_\theta)$ y ruido gaussiano, $y\sim\mathcal{N}(0,K_y)$ y*

$$\log p(y\mid\theta)=-\tfrac12 y^\top K_y^{-1}y-\tfrac12\log\det K_y-\tfrac{n}{2}\log 2\pi. \tag{3.1}$$

Su gradiente es $\dfrac{\partial}{\partial\theta_j}\log p(y\mid\theta)=\dfrac12\operatorname{tr}\!\Big((\alpha\alpha^\top-K_y^{-1})\dfrac{\partial K_y}{\partial\theta_j}\Big)$, *con* $\alpha=K_y^{-1}y$.

Los tres términos de (3.1) tienen lectura: el primero mide el ajuste a los datos, el segundo penaliza la complejidad (una ℓ pequeña hace K_y casi diagonal y aumenta $\log\det K_y$) y el

tercero es una constante. Esta compensación automática entre ajuste y complejidad es la navaja de Occam bayesiana [38, cap. 5].

Observación 3.9 (Cálculo estable y óptimos locales). En la práctica nunca se invierte K_y. Se calcula su factorización de Cholesky $K_y = LL^\top$, se resuelven dos sistemas triangulares para $\alpha = L^{-\top}L^{-1}y$, se usa $\log\det K_y = 2\sum_i \log L_{ii}$ y $\sigma_n^2(x) = k(x,x) - \|L^{-1}k_*\|^2$; el coste es $O(n^3)$ una vez y $O(n^2)$ por predicción [38, cap. 2]. Si dos puntos casi coinciden, K_y es casi singular y se añade un pequeño término diagonal (*jitter*, del orden de $10^{-6}\sigma_f^2$; ejercicio 3.5). La función (3.1) tiene a menudo varios máximos locales: uno con ℓ pequeña que explica las fluctuaciones como señal y otro con ℓ grande que las atribuye al ruido; por eso se reinicia el optimizador desde varios puntos. La tarea 1 del curso 4F13 de Cambridge [9] pide precisamente encontrarlos e interpretarlos.

3.5 Redes neuronales como sustitutos

Un proceso gaussiano exacto escala como $O(n^3)$ y sufre con más de unas decenas de variables. Cuando hay miles de datos (históricos de planta, simulaciones baratas) o entradas de alta dimensión, un perceptrón multicapa (módulo 9) es una alternativa natural, pero no da incertidumbre por sí solo.

Definición 3.11 (*Ensemble* profundo). Se entrenan M redes con inicializaciones distintas (y, opcionalmente, remuestreos de los datos); la red m predice una media $\mu_m(x)$ y, si se entrena con la verosimilitud gaussiana, una varianza $\sigma_m^2(x)$. La predicción del *ensemble* es la mezcla uniforme $\frac{1}{M}\sum_m \mathcal{N}(\mu_m(x), \sigma_m^2(x))$ [27].

Proposición 3.12 (Media y varianza de la mezcla). *La mezcla tiene media* $\mu(x) = \frac{1}{M}\sum_m \mu_m(x)$ *y varianza*

$$\sigma^2(x) = \frac{1}{M}\sum_m \big(\sigma_m^2(x) + \mu_m^2(x)\big) - \mu^2(x) = \underbrace{\frac{1}{M}\sum_m \sigma_m^2(x)}_{\text{aleatoria}} + \underbrace{\frac{1}{M}\sum_m \big(\mu_m(x) - \mu(x)\big)^2}_{\text{epistémica}}.$$

El sumando epistémico (desacuerdo entre redes) crece lejos de los datos, que es lo que necesita la optimización. Garnett [19, §8.11] discute estos y otros modelos alternativos al proceso gaussiano.

3.6 Redes informadas por la física

Cuando se conoce la ecuación que gobierna parte del proceso, puede incorporarse al entrenamiento. Una *red informada por la física* (PINN) [37] minimiza

$$\mathcal{L}(\theta, \kappa) = \frac{1}{n}\sum_{i=1}^{n}\big(T_\theta(t_i) - T_i\big)^2 + \frac{\lambda}{m}\sum_{l=1}^{m}\mathcal{R}_{\theta,\kappa}(\tau_l)^2,$$

donde el primer término ajusta las medidas y el segundo penaliza el residuo $\mathcal{R}$ de la ecuación diferencial en m puntos de colocación τ_l. Las derivadas de T_θ respecto a su entrada se obtienen por diferenciación automática (módulo 10, tema 3), y los parámetros físicos desconocidos κ se aprenden a la vez que θ (problema inverso).

3.7 Caso de estudio: tres sustitutos del coste

Herramienta	Para qué sirve	Licencia	Cuándo elegirla
scikit-learn, `GaussianProcessRegressor` [40]	PG exacto con núcleos combinables y máxima verosimilitud marginal con reinicios	BSD	Hasta unos miles de puntos, prototipos
GPyTorch [18]	PG sobre PyTorch con álgebra lineal iterativa y GPU	MIT	Muchos datos, modelos a medida, BoTorch
BoTorch [5] (modelos)	PG de una y varias salidas, multifidelidad, con heterocedasticidad	MIT	Sustituto dentro de un bucle de optimización
PyTorch	Perceptrones, *ensembles*, PINN	BSD	Muchos datos, entradas de alta dimensión, física conocida
scikit-learn (lineales, polinomios)	Regresión de bases y validación cruzada	BSD	Pocos datos y respuesta suave

Cuadro 3.1: Tecnologías para construir modelos sustitutos.

Tema 4

Optimización bayesiana

La optimización bayesiana usa el sustituto probabilístico del tema 3 para decidir el siguiente experimento: elige el punto que maximiza una *función de adquisición*, una medida del valor esperado de ensayarlo según el modelo. El método procede de Jones, Schonlau y Welch [23], que introdujeron el algoritmo EGO con la mejora esperada; la exposición sigue la §4 del tutorial de Frazier [15], los capítulos 7, 8 y 11 de Garnett [19], la revisión de Shahriari *et al.* [41] y el capítulo 16 de Kochenderfer y Wheeler [26]. Todo el tema trata problemas de *minimización*; para maximizar basta cambiar el signo de f.

4.1 El algoritmo

Algoritmo 4.1 (Optimización bayesiana). Datos: dominio $\mathcal{X}$, presupuesto N, diseño inicial de n_0 puntos.

1. Evaluar el diseño inicial (por ejemplo, un LHS con $n_0 \approx 10\,d$ o menos si el presupuesto es corto) y formar $\mathcal{D}_{n_0}$.
2. Para $n = n_0, \dots, N-1$:
   (a) ajustar el sustituto a $\mathcal{D}_n$ (hiperparámetros por verosimilitud marginal);
   (b) calcular $x_{n+1} \in \arg\max_{x \in \mathcal{X}} \alpha_n(x)$ con un optimizador auxiliar (varios arranques de L-BFGS-B o una nube de candidatos);
   (c) evaluar $y_{n+1} = f(x_{n+1}) + \varepsilon$ y añadir el par a los datos.
3. Recomendar $\hat{x}_N \in \arg\min_x \mu_N(x)$ (entre los puntos evaluados o en todo $\mathcal{X}$) y confirmar con ensayos repetidos.

La función de adquisición debe equilibrar dos tendencias: *explotar* (ensayar donde la media μ_n es baja) y *explorar* (ensayar donde la incertidumbre σ_n es grande). En lo que sigue, $\mu = \mu_n(x)$, $\sigma = \sigma_n(x)$ es la desviación típica de la función latente $f(x)$ (no de una nueva observación) y f_n^* es el valor de referencia: el mínimo observado si no hay ruido o el mínimo de μ_n en los puntos evaluados si lo hay [15, §5].

4.2 Probabilidad de mejora

Definición 4.2 (Probabilidad de mejora). $\mathrm{PI}_n(x) = \Pr\big[f(x) < f_n^* - \xi\big] = \Phi\Big(\dfrac{f_n^* - \xi - \mu}{\sigma}\Big)$, con un margen $\xi \geq 0$.

La igualdad se debe a que $f(x) \sim \mathcal{N}(\mu, \sigma^2)$ y a la tipificación (módulo 0.8, §6.3). La PI ignora *cuánto* se mejora: prefiere una mejora casi segura y diminuta a una probable y grande, y con $\xi = 0$ tiende a muestrear junto al mejor punto actual [19, §7.5].

4.3 Mejora esperada

Definición 4.3 (Mejora y mejora esperada). La *mejora* en x es $I(x) = \max\big(f_n^* - f(x), 0\big)$, y la *mejora esperada* es $\mathrm{EI}_n(x) = \mathbb{E}_n[I(x)]$, donde la esperanza se toma respecto a la posterior de $f(x)$ dados los datos [23, ec. (14)].

Teorema 4.4 (Forma cerrada de la mejora esperada). *Si* $f(x) \mid \mathcal{D}_n \sim \mathcal{N}(\mu, \sigma^2)$ *con* $\sigma > 0$ *y* $z = (f_n^* - \mu)/\sigma$*, entonces*

$$\mathrm{EI}_n(x) = (f_n^* - \mu)\,\Phi(z) + \sigma\,\varphi(z).$$

Si $\sigma = 0$*,* $\mathrm{EI}_n(x) = \max(f_n^* - \mu, 0)$*.*

Frazier [15, ec. (8)] da una forma equivalente para maximización, escrita con $\Delta = \mu - f_n^*$ y $[\Delta]^+$; Jones *et al.* [23, ec. (15)] la dan en la forma del teorema.

Proposición 4.5 (Propiedades de la mejora esperada). *Como función de* (μ, σ)*, con* $\sigma > 0$*:*

(i) $\mathrm{EI} > 0$ *y* $\mathrm{EI} \geq \max(f_n^* - \mu, 0)$*;*

(ii) $\partial\,\mathrm{EI}\,/\partial\mu = -\Phi(z) < 0$*: a igual incertidumbre, se prefiere la media más baja;*

(iii) $\partial\,\mathrm{EI}\,/\partial\sigma = \varphi(z) > 0$*: a igual media, se prefiere la mayor incertidumbre.*

4.4 Cotas de confianza y muestreo de Thompson

Definición 4.7 (Cota inferior de confianza). Para un parámetro $\beta_n > 0$, se elige $x_{n+1} \in \arg\min_x \mathrm{LCB}_n(x)$, con $\mathrm{LCB}_n(x) = \mu_n(x) - \sqrt{\beta_n}\,\sigma_n(x)$. En maximización se usa la cota superior $\mathrm{UCB}_n = \mu_n + \sqrt{\beta_n}\sigma_n$ (GP-UCB).

Srinivas *et al.* [44] demostraron que, con una sucesión β_n que crece logarítmicamente, el regret acumulado de GP-UCB crece como $O^*\big(\sqrt{N\beta_N\gamma_N}\big)$, donde γ_N es la máxima información que N observaciones ruidosas pueden aportar sobre f; para el núcleo exponencial cuadrático, γ_N crece como una potencia de $\log N$, y el regret medio R_N/N tiende a cero. La teoría da una justificación al equilibrio entre explorar y explotar, pero los valores de β_n que exige son conservadores; en la práctica se usan constantes entre 1 y 9 [19, §8.4 y cap. 10].

El *muestreo de Thompson* extrae una función $\tilde{f} \sim \mathcal{GP}(\mu_n, k_n)$ de la posterior y ensaya su mínimo; explora en proporción a la probabilidad de que cada punto sea el óptimo y se paraleliza con facilidad [19, §7.9].

4.5 Optimizar la función de adquisición

La función α_n es barata pero multimodal: vale casi cero en la mayor parte del dominio y tiene picos estrechos. Lo habitual es evaluarla en una nube de miles de candidatos (aleatorios y perturbaciones de los mejores puntos) y refinar los mejores con L-BFGS-B, usando su gradiente analítico o por diferenciación automática [19, §9.2], [5]. Con valores de EI muy pequeños conviene trabajar con su logaritmo para evitar regiones numéricamente planas.

Observación 4.8 (Varianza latente y varianza predictiva). Muchas bibliotecas devuelven la desviación típica de una nueva *observación*, $\sqrt{\sigma_n^2(x) + \sigma_\varepsilon^2}$, en lugar de la de $f(x)$. Usarla en

la EI o en la probabilidad de cumplir una restricción es un error: en un punto ya ensayado la incertidumbre nunca baja del nivel de ruido, la EI no se anula y el algoritmo repite indefinidamente el mismo ajuste. En la práctica, con `GaussianProcessRegressor` y un `WhiteKernel`, hay que restar la varianza del ruido. En los experimentos de la práctica con la inyectora, sin esta corrección el algoritmo dedicaba la segunda mitad del presupuesto a repetir ajustes casi idénticos en una esquina del dominio.

4.6 Restricciones

Proposición 4.9 (Mejora esperada con restricciones). *Sea el problema de la definición 1.7 con una restricción $c(x) \le 0$ modelada con un proceso gaussiano independiente del de f, con posterior $\mathcal{N}(\mu_c, \sigma_c^2)$, y sea f_n^* el mejor valor* factible*. La mejora restringida $I_c(x) = I(x)\,\mathbf{1}[c(x) \le 0]$ tiene esperanza*

$$\mathrm{EI}_c(x) = \mathrm{EI}_n(x) \cdot \Phi\Big(\frac{-\mu_c(x)}{\sigma_c(x)}\Big).$$

Gardner *et al.* [17] y Gelbart, Snoek y Adams [20] estudian la optimización bayesiana con restricciones desconocidas colocando también un proceso gaussiano sobre cada restricción; el segundo admite además restricciones que se evalúan por separado del objetivo. El producto de la proposición 4.9 es la forma más sencilla de combinar ambos modelos [19, §11.2]. Para una especificación $w(x) \le w_{\max}$, la probabilidad de cumplirla es $\Phi\big((w_{\max} - \mu_w)/\sigma_w\big)$. Si todavía no hay ningún punto factible, se maximiza solo la probabilidad de factibilidad [19, §11.2]. Las restricciones conocidas y baratas no se modelan: se imponen al generar los candidatos.

4.7 Evaluaciones por lotes

En una planta se suelen hacer varios ensayos por turno, o hay varias máquinas: se necesita proponer q puntos a la vez. La generalización natural es la *mejora esperada multipunto* [21],

$$q\text{-}\mathrm{EI}(x^{(1)}, \dots, x^{(q)}) = \mathbb{E}_n\Big[\max\big(f_n^* - \min_j f(x^{(j)}), 0\big)\Big],$$

que no tiene forma cerrada útil y se estima por Monte Carlo con la distribución conjunta de los q valores; BoTorch la optimiza con gradientes de Monte Carlo [5]. Dos heurísticas secuenciales más sencillas, de Ginsbourger, Le Riche y Carraro [21], eligen los puntos uno a uno añadiendo un dato ficticio en cada punto elegido: el *creyente del kriging* usa como valor la media posterior $\mu_n(x^{(j)})$, y el *mentiroso constante* usa un valor fijo L (el mínimo, la media o el máximo observado; cuanto mayor es L, más se dispersan los puntos). Como la varianza posterior no depende de los valores (observación 3.6), el dato ficticio reduce la incertidumbre alrededor de $x^{(j)}$ y aleja de él al siguiente punto del lote. Si los ensayos terminan en momentos distintos, los pendientes se tratan igual (optimización asíncrona [19, §11.4]).

4.8 Varios objetivos

En la inyectora se puede querer, en lugar de un único coste, minimizar a la vez la fracción defectuosa y el tiempo de ciclo, y dejar la elección final a la dirección de la planta.

Definición 4.10 (Dominancia y frente de Pareto). Sean $F(x) = (F_1(x), \dots, F_m(x))$ objetivos a minimizar. El punto x *domina* a x' si $F_i(x) \le F_i(x')$ para todo i y $F_i(x) < F_i(x')$ para algún i. Un punto es *óptimo de Pareto* si ningún otro lo domina; la imagen del conjunto de óptimos es el *frente de Pareto* [26, §12.1], [31, §9.2].

Definición 4.11 (Hipervolumen). Dado un punto de referencia r dominado por todos los puntos de interés, el *hipervolumen* de un conjunto P de vectores objetivo es la medida (área en dos dimensiones) de la región $\{v : \exists p \in P,\ p \le v \le r\}$ dominada por P y acotada por r.

El hipervolumen crece si y solo si se añade un punto no dominado por el conjunto, y por eso es la medida habitual de progreso. La *mejora esperada del hipervolumen* (EHVI) es la generalización de la EI; Daulton, Balandat y Bakshy [11] la extienden a lotes con restricciones (qEHVI) y la calculan con diferenciación automática. Una alternativa más sencilla es escalarizar.

Proposición 4.12 (Escalarización de Chebyshev). *Sean $w_i > 0$ pesos y $z^\star$ un punto ideal con $z_i^\star \le F_i(x)$ para todo x. Todo minimizador de $s_w(x) = \max_i w_i\big(F_i(x) - z_i^\star\big)$ es débilmente óptimo de Pareto: no existe x' con $F_i(x') < F_i(x)$ para todo i.*

ParEGO [25] aplica la EI a la escalarización de Chebyshev aumentada con pesos aleatorios distintos en cada iteración, y así recorre el frente. A diferencia de la suma ponderada, la escalarización de Chebyshev alcanza también los puntos de las partes no convexas del frente (ejercicio 4.8).

4.9 Varias fidelidades

A menudo hay versiones más baratas e imprecisas del experimento: una simulación del llenado del molde antes del ensayo en máquina, un ensayo de resistencia a 7 días en lugar de 28, un entrenamiento corto en lugar de uno completo. La optimización multifidelidad modela conjuntamente $f(x, s)$, con s la fidelidad, y elige a la vez el punto y la fidelidad [19, §11.5], [15, §5]. La regla más simple es dividir la adquisición por el coste: Snoek, Larochelle y Adams [43] proponen la mejora esperada por segundo, $\mathrm{EI}(x)/c(x)$, con el coste $c(x)$ también modelado. Kandasamy *et al.* [24] (MF-GP-UCB) usan las fidelidades baratas para descartar regiones poco prometedoras y reservan las caras para las regiones buenas, con garantías de regret mejores que las de ignorar las aproximaciones.

4.10 Caso de estudio: el optimizador de la inyectora

Herramienta	Para qué sirve	Licencia	Cuándo elegirla
BoTorch [5]	Adquisiciones de Monte Carlo: qEI, qNEI, qEHVI, multifidelidad, restricciones	MIT	Problemas a medida, lotes, varios objetivos
Ax [4]	Interfaz de alto nivel sobre BoTorch: experimentos, restricciones de resultado, ensayos pendientes	MIT	Campañas reales con varias personas y máquinas
Optuna `GPSampler` [34]	Optimización bayesiana con PG dentro de Optuna	MIT	Si ya se usa Optuna
SMAC3 [29]	BO con bosques aleatorios y espacios condicionales	BSD	Variables categóricas y jerárquicas
scikit-learn + SciPy	PG y optimización de la adquisición escritos a mano	BSD	Docencia y prototipos (práctica)

Cuadro 4.1: Tecnologías para la optimización bayesiana.

Tema 5

Casos de estudio: moldeo por inyección y hormigón

Este tema reúne las piezas de los anteriores en dos casos completos: el ajuste de la inyectora, que se ha ido desarrollando desde el tema 1, y la formulación de un hormigón con menos emisiones de CO_2. Ambos usan datos sintéticos generados por los simuladores de la Parte 5 de la práctica; la revisión de Fernandes *et al.* [13] sobre modelado y optimización del moldeo por inyección y el trabajo de Ament *et al.* [2] sobre hormigón sostenible con optimización bayesiana son las referencias de los procesos reales. La metodología sigue las guías de Montgomery [33, §1.5] y Garnett [19, cap. 9].

5.1 Del problema al optimizador

Algoritmo 5.1 (Lista de diseño de un optimizador experimental).

1. *Objetivo*: una cifra con unidades y su justificación económica (coste por pieza buena, emisiones por m^3). Si hay varias, decidir si se escalariza o se busca el frente de Pareto.
2. *Variables*: las que se pueden fijar en planta, con rangos seguros acordados con quien opera el proceso; escala lineal o logarítmica.
3. *Restricciones*: clasificarlas (conocidas, caras, ocultas, de seguridad) según el tema 1.
4. *Ruido*: de dónde viene, cuánto vale (réplicas o un estudio previo) y si es heterocedástico.
5. *Presupuesto y logística*: número de ensayos, cuántos en paralelo, cuánto tarda cada uno y si hay fidelidades más baratas.
6. *Cribado*: un factorial o una fracción para descartar variables y detectar interacciones.
7. *Bucle*: diseño inicial, sustituto, adquisición, recomendación (algoritmo 4.1).
8. *Parada y confirmación*: criterio de parada (tema 6) y ensayos repetidos en la recomendación.
9. *Registro*: guardar cada ensayo con su contexto (lote de material, turno, operario) para poder reutilizarlo.

La comparación con la práctica tradicional de variar un factor cada vez (OFAT) muestra por qué se necesita todo lo anterior.

5.2 Caso A: el ajuste completo de la inyectora

5.3 Caso B: un hormigón con menos CO_2

El cemento es responsable de una fracción importante de las emisiones globales de CO_2, lo que motiva sustituir parte de él por adiciones como las cenizas volantes [2]. La dificultad es que la resistencia, que se mide rompiendo probetas a los 28 días, baja con la sustitución.

Definición 5.4 (Resistencia característica y resistencia media objetivo). La resistencia característica f_{ck} es el cuantil del 5 % de la resistencia a compresión a 28 días. Si la resistencia de las amasadas de una mezcla es normal con media f_{cm} y desviación σ_p (variabilidad de producción), $f_{ck} = f_{cm} - 1{,}645\,\sigma_p$, y para garantizar f_{ck} la mezcla debe tener $f_{cm} \geq f_{ck} + 1{,}645\,\sigma_p$.

La fórmula es el cuantil de la normal (módulo 0.8, §6.3): $\Pr[f < f_{cm} - 1{,}645\sigma_p] = \Phi(-1{,}645) = 0{,}05$.

Observación 5.6 (Objetivo conocido, restricción desconocida). Cuando el objetivo g es una fórmula conocida y lo caro es la restricción, la mejora en un candidato es determinista, $I(x) = \max(g^* - g(x), 0)$, y la EI restringida de la proposición 4.9 se reduce a

$$\alpha(x) = \max\big(g^* - g(x), 0\big) \cdot \Pr\big[f_{28}(x) \geq 36{,}58\big] = \max\big(g^* - g(x), 0\big) \cdot \Phi\Big(\frac{\mu_{28}(x) - 36{,}58}{\sigma_{28}(x)}\Big).$$

Solo hay que modelar lo que no se conoce. La misma idea mejora el caso de la inyectora, cuyo tiempo de ciclo es conocido: basta modelar la fracción defectuosa.

Herramienta	Para qué sirve	Licencia o tipo	Cuándo elegirla
Ax, interfaz `Client` [4]	Pedir ensayos (`get_next_trials`), registrar resultados (`complete_trial`), restricciones de resultado, ensayos pendientes	MIT	Campañas en planta o laboratorio con varias personas
BoTorch [5]	Adquisiciones con objetivos conocidos, restricciones y multifidelidad	MIT	Formulaciones a medida como la observación 5.6
pyDOE3, `scipy.stats.qmc`	Cribado y diseño inicial	BSD	Fases 6 y 7 de la lista
Sustainable-Concrete [2]	Código de los modelos y del bucle de Ament *et al.*	Código abierto (GitHub)	Punto de partida para mezclas reales
Hoja de cálculo o base de datos	Registro de ensayos con contexto	—	Siempre: es el activo más valioso

Cuadro 5.1: Tecnologías para desplegar un optimizador experimental.

Tema 6

Ajuste de hiperparámetros, validación y parada

Elegir la tasa de aprendizaje, la regularización o la arquitectura de un modelo es un problema de caja negra con las mismas características que el ajuste de una inyectora: cada evaluación es un entrenamiento caro, el resultado es ruidoso (depende de la semilla y de la partición de los datos) y el presupuesto es limitado. Tiene además dos rasgos propios: espacios mixtos y condicionales, y evaluaciones parciales baratas (pocas épocas, un subconjunto de los datos). Este tema sigue el capítulo 1 del libro de AutoML de Hutter, Kotthoff y Vanschoren, escrito por Feurer y Hutter [14], y los artículos originales de la búsqueda aleatoria [6], TPE [7], Hyperband [28] y ASHA [3]. Cierra el módulo con dos preguntas comunes a todos los casos: cómo validar el sustituto y cuándo dejar de experimentar.

6.1 El ajuste de hiperparámetros como optimización de caja negra

Definición 6.1 (Problema de ajuste de hiperparámetros). Sea $\mathcal{A}_\lambda$ un algoritmo de aprendizaje con configuración $\lambda \in \Lambda$, entrenado con D_{ent}. El problema es

$$\min_{\lambda \in \Lambda} \; \mathcal{L}\big(\mathcal{A}_\lambda(D_{\text{ent}}), D_{\text{val}}\big),$$

donde $\mathcal{L}$ es la pérdida de validación (o la media de una validación cruzada, módulo 8, §6.5). El espacio Λ puede mezclar variables continuas, enteras y categóricas, y tener variables *condicionales* (el momento solo existe si el optimizador es SGD) [14, §1.2].

Las variables que abarcan varios órdenes de magnitud (tasa de aprendizaje, regularización) se buscan en escala logarítmica. Las evaluaciones parciales (un entrenamiento de r épocas en lugar de R) son fidelidades bajas, y su aprovechamiento es el origen de los métodos de la §6.4 [14, §1.4]. Las leyes de escalado del módulo 16 son la versión extrema de esta idea para los LLM: se ajustan hiperparámetros y curvas en modelos pequeños y se extrapolan.

6.2 Búsqueda aleatoria frente a rejilla

Proposición 6.3 (Garantía de la búsqueda aleatoria). *Sea $\Lambda_q \subset \Lambda$ la región de las configuraciones del mejor q (en probabilidad bajo la distribución de muestreo). Con n configuraciones independientes,* $\Pr[\text{alguna cae en } \Lambda_q] = 1 - (1-q)^n$. *En consecuencia, para garantizarlo con probabilidad p basta*

$$n \geq \frac{\log(1-p)}{\log(1-q)}.$$

Con $q = 0{,}05$ y $p = 0{,}95$ salen $n \geq 58{,}4$: 59 ensayos, independientemente de la dimensión. La garantía es sobre cuantiles del espacio, no sobre la distancia al óptimo, pero explica por qué la búsqueda aleatoria es una línea base difícil de batir. Bergstra y Bengio [6] añaden el argumento de la *dimensión efectiva baja*: en la mayoría de los problemas solo unos pocos hiperparámetros importan, y cuáles importan cambia de un conjunto de datos a otro. Una rejilla de $n = m^d$ puntos solo prueba $m = n^{1/d}$ valores de cada hiperparámetro; n puntos aleatorios prueban n valores distintos de cada uno (la misma propiedad de proyección del hipercubo latino, tema 2).

6.3 El estimador de Parzen con estructura de árbol (TPE)

La optimización bayesiana con procesos gaussianos modela $p(y \mid \lambda)$. TPE modela al revés: la distribución de las configuraciones condicionada a que sean buenas o malas [7, §4].

Definición 6.4 (TPE). Sea y^* el cuantil γ de las pérdidas observadas, de modo que $\Pr[y < y^*] = \gamma$. Se definen dos densidades sobre Λ, estimadas con núcleos (estimadores de Parzen) a partir de las configuraciones observadas:

$$p(\lambda \mid y) = \begin{cases} \ell(\lambda) & \text{si } y < y^* \text{ (configuraciones buenas)}, \\ g(\lambda) & \text{si } y \geq y^* \text{ (el resto)}. \end{cases}$$

En espacios condicionales, cada hiperparámetro tiene su propia densidad unidimensional y la estructura de árbol decide cuáles intervienen.

Proposición 6.5 (TPE maximiza la mejora esperada). *Con la mejora respecto a y^*, $\mathrm{EI}_{y^*}(\lambda) = \int_{-\infty}^{y^*} (y^* - y)\, p(y \mid \lambda)\, dy$, el modelo de la definición 6.4 da*

$$\mathrm{EI}_{y^*}(\lambda) \;\propto\; \left(\gamma + \frac{g(\lambda)}{\ell(\lambda)}(1-\gamma)\right)^{-1},$$

con una constante de proporcionalidad que no depende de λ. Maximizar la EI equivale a maximizar el cociente $\ell(\lambda)/g(\lambda)$.

En la práctica TPE muestrea muchos candidatos de ℓ y se queda con el de mayor cociente ℓ/g. Su coste es lineal en el número de observaciones, maneja con naturalidad variables categóricas y condicionales, y es el muestreador por defecto de Optuna [34]. Bergstra *et al.* [7, §6] usan $\gamma = 0{,}15$ en sus experimentos.

6.4 Evaluaciones parciales: Successive Halving y Hyperband

Definición 6.6 (Successive Halving). Dados n configuraciones, un recurso mínimo r (épocas, iteraciones, fracción de datos) y un factor $\eta > 1$ (habitualmente 3): se entrenan todas con r, se conserva la mejor fracción $1/\eta$, se multiplica el recurso por η y se repite hasta que queda una configuración o se alcanza el recurso máximo R.

El método supone que las configuraciones malas se reconocen pronto. Si se fija el presupuesto total, hay que elegir entre muchas configuraciones con poco recurso (riesgo de descartar las que empiezan despacio) o pocas con mucho (poca exploración). Hyperband no elige: ejecuta varias versiones [28, §3.2].

Algoritmo 6.7 (Hyperband). Datos: recurso máximo R y η. Sean $s_{\max} = \lfloor \log_\eta R \rfloor$ y $B = (s_{\max}+1)R$. Para $s = s_{\max}, s_{\max}-1, \dots, 0$ (cada valor es un *bracket*):

1. $n = \left\lceil \frac{B}{R} \frac{\eta^s}{s+1} \right\rceil$ configuraciones aleatorias y recurso inicial $r = R\eta^{-s}$;
2. Successive Halving en $s+1$ rondas: en la ronda $i = 0, \dots, s$, $n_i = \lfloor n\eta^{-i} \rfloor$ configuraciones con recurso $r_i = r\eta^i$; se conservan las $\lfloor n_i/\eta \rfloor$ mejores.

Se devuelve la mejor configuración evaluada con el recurso completo.

Proposición 6.8 (Presupuesto de un bracket). *En el bracket s, cada ronda consume como mucho $nr \le \frac{B}{s+1} + R\eta^{-s}$ unidades de recurso, y el bracket completo, como mucho $B + (s+1)R\eta^{-s}$. Todos los brackets cuestan, pues, aproximadamente B, y un ciclo de Hyperband, unas $(s_{\max}+1)B$.*

Hyperband es síncrono: cada ronda espera a que terminen todas sus configuraciones. ASHA [3] lo hace asíncrono: un trabajador libre promueve a la siguiente ronda cualquier configuración que esté entre la mejor fracción $1/\eta$ de las ya completadas en la suya, o empieza una nueva; así escala linealmente con cientos de trabajadores. Optuna implementa ambos como *pruners*, y BOHB combina Hyperband con un modelo tipo TPE para elegir las configuraciones [14, §1.4.2].

6.5 Validación del sustituto

Antes de confiar en las recomendaciones de un sustituto conviene comprobar que predice bien y que sus intervalos están calibrados. Para un proceso gaussiano, la validación cruzada dejando uno fuera no exige reajustar el modelo.

Teorema 6.10 (Validación cruzada exacta de un proceso gaussiano). *Con hiperparámetros fijos, la distribución predictiva de la observación y_i dados los demás datos es normal con*

$$\mu_{-i} = y_i - \frac{[K_y^{-1} y]_i}{[K_y^{-1}]_{ii}}, \qquad \sigma^2_{-i} = \frac{1}{[K_y^{-1}]_{ii}}.$$

Rasmussen y Williams [38, cap. 5] dan el mismo resultado y lo usan como alternativa a la verosimilitud marginal para elegir hiperparámetros. Con las predicciones LOO se calculan los *residuos tipificados* $r_i = (y_i - \mu_{-i})/\sigma_{-i}$: si el modelo es correcto, se comportan aproximadamente como normales estándar, con media próxima a 0, desviación próxima a 1 y un 5 % fuera de ± 2. Una desviación mucho mayor que 1 revela un modelo demasiado confiado (intervalos estrechos); mucho menor, uno demasiado cauto.

6.6 Cuándo dejar de experimentar

No existe una regla universal; Garnett [19, §5.4 y §9.3] trata la parada como una decisión más, que compara el valor esperado de seguir con su coste. Las reglas habituales son:

(i) *Presupuesto agotado*: la más común y la única que garantiza el plazo.

(ii) *Mejora esperada pequeña*: Jones, Schonlau y Welch [23, §4.2] detienen EGO cuando la máxima EI es menor que el 1 % del mejor valor actual. Con ruido es engañosa: en el caso de la inyectora se activa en el ensayo 13 de 30, cuando la recomendación aún puede

mejorar mucho, porque la EI calculada con la varianza latente es pequeña en todas partes salvo en regiones que la adquisición todavía no ha encontrado.

(iii) *Cota de regret*: si las bandas de confianza son válidas, la distancia al óptimo de la recomendación está acotada (proposición 6.12). Makarova *et al.* [30] proponen parar cuando esta cota es menor que el error estadístico con que se estima la propia pérdida de validación: seguir optimizando sería ajustar el ruido.

(iv) *Criterio económico*: parar cuando la mejora esperada por unidad de producto multiplicada por el volumen de producción sea menor que el coste de un ensayo más.

Proposición 6.12 (Cota del regret simple). *Si con probabilidad al menos* $1-\delta$ *se cumple* $\mathrm{LCB}_n(x) \le f(x) \le \mathrm{UCB}_n(x)$ *para todo* $x \in \mathcal{X}$*, entonces, con la misma probabilidad, para cualquier recomendación* $\hat{x}$*,*

$$f(\hat{x}) - \min_{x\in\mathcal{X}} f(x) \le \mathrm{UCB}_n(\hat{x}) - \min_{x\in\mathcal{X}} \mathrm{LCB}_n(x).$$

La cota es pequeña cuando el modelo está seguro del valor en la recomendación y ningún otro punto tiene una cota inferior mucho más baja: es decir, cuando ya no queda ninguna región sin explorar que pudiera esconder algo mejor.

6.7 Caso de estudio: ajuste de un *gradient boosting*

Herramienta	Para qué sirve	Licencia	Cuándo elegirla
Optuna [34]	TPE, GP, CMA-ES, NSGA-II; *pruners* `SuccessiveHalvingPruner`, `HyperbandPruner`, `MedianPruner`	MIT	Opción por defecto en Python, espacios definidos al ejecutar
SMAC3 [29]	BO con bosques aleatorios, Hyperband, configuración sobre varias instancias	BSD	Espacios condicionales, configuración de algoritmos
Ax y BoTorch [4, 5]	BO con GP, multiobjetivo y parada temprana	MIT	Pocos ensayos muy caros, varios objetivos
scikit-learn	`RandomizedSearchCV` y búsqueda por reducción sucesiva	BSD	Modelos de scikit-learn con validación cruzada
Google Vizier [22]	Servicio de optimización de caja negra con parada temprana y transferencia	Servicio	Referencia de diseño de un servicio a escala

Cuadro 6.1: Tecnologías para el ajuste de hiperparámetros.

Bibliografía

[1] Stanford University. AA222/CS361: Engineering Design Optimization, primavera de 2026. Libro de texto: Kochenderfer y Wheeler [26]. https://aa222.stanford.edu/

[2] S. Ament, A. Witte, N. Garg y J. Kusuma. Sustainable concrete via Bayesian optimization. Prepublicación, 2023. https://arxiv.org/abs/2310.18288

[3] L. Li, K. Jamieson, A. Rostamizadeh, E. Gonina, M. Hardt, B. Recht y A. Talwalkar. A system for massively parallel hyperparameter tuning. En *Proceedings of Machine Learning and Systems (MLSys)*, 2020. https://arxiv.org/abs/1810.05934

[4] Meta Platforms. Ax: Adaptive Experimentation Platform. Licencia MIT. https://ax.dev/ y https://github.com/facebook/Ax

[5] M. Balandat, B. Karrer, D. R. Jiang, S. Daulton, B. Letham, A. G. Wilson y E. Bakshy. BoTorch: a framework for efficient Monte-Carlo Bayesian optimization. Prepublicación, 2019 (versión de 2020). Licencia del código: MIT. https://arxiv.org/abs/1910.06403

[6] J. Bergstra e Y. Bengio. Random search for hyper-parameter optimization. *Journal of Machine Learning Research*, 13:281–305, 2012.

[7] J. Bergstra, R. Bardenet, Y. Bengio y B. Kégl. Algorithms for hyper-parameter optimization. En *Advances in Neural Information Processing Systems 24*, 2011.

[8] G. E. P. Box y K. B. Wilson. On the experimental attainment of optimum conditions. *Journal of the Royal Statistical Society, Series B*, 13(1):1–45, 1951.

[9] C. E. Rasmussen. 4F13: Probabilistic Machine Learning. University of Cambridge, Michaelmas de 2024. Tarea 1, «Gaussian Processes». https://mlg.eng.cam.ac.uk/teaching/4f13/2425/

[10] Y. Chen, A. Huang, Z. Wang, I. Antonoglou, J. Schrittwieser, D. Silver y N. de Freitas. Bayesian optimization in AlphaGo. Prepublicación, 2018. https://arxiv.org/abs/1812.06855

[11] S. Daulton, M. Balandat y E. Bakshy. Differentiable expected hypervolume improvement for parallel multi-objective Bayesian optimization. Prepublicación, 2020. https://arxiv.org/abs/2006.05078

[12] R. Evans y J. Gao. DeepMind AI reduces Google data centre cooling bill by 40 %. Blog de Google DeepMind, 20 de julio de 2016. https://deepmind.google/blog/deepmind-ai-reduces-google-data-centre-cooling-bill-by-40/

[13] C. Fernandes *et al.* Modeling and optimization of the injection-molding process: a review. *Advances in Polymer Technology*, 2018 (publicado en línea en 2016). https://doi.org/10.1002/adv.21683

[14] M. Feurer y F. Hutter. Hyperparameter optimization. En F. Hutter, L. Kotthoff y J. Vanschoren (eds.), *Automated Machine Learning: Methods, Systems, Challenges*, cap. 1, págs. 3–33. Springer, 2019. https://www.automl.org/book/

[15] P. I. Frazier. A tutorial on Bayesian optimization. Prepublicación, 2018. https://arxiv.org/abs/1807.02811

[16] J. Gao. Machine learning applications for data center optimization. Informe técnico, Google, 2014. https://research.google/pubs/machine-learning-applications-for-data-center-optimization/

[17] J. Gardner, M. Kusner, Z. Xu, K. Weinberger y J. Cunningham. Bayesian optimization with inequality constraints. En *Proceedings of the 31st International Conference on Machine Learning*, PMLR 32(2):937–945, 2014.

[18] J. R. Gardner, G. Pleiss, D. Bindel, K. Q. Weinberger y A. G. Wilson. GPyTorch: blackbox matrix-matrix Gaussian process inference with GPU acceleration. Prepublicación, 2018. https://arxiv.org/abs/1809.11165

[19] R. Garnett. *Bayesian Optimization*. Cambridge University Press, 2023. https://bayesoptbook.com/

[20] M. A. Gelbart, J. Snoek y R. P. Adams. Bayesian optimization with unknown constraints. Prepublicación, 2014. https://arxiv.org/abs/1403.5607

[21] D. Ginsbourger, R. Le Riche y L. Carraro. Kriging is well-suited to parallelize optimization. En Y. Tenne y C. K. Goh (eds.), *Computational Intelligence in Expensive Optimization Problems*, págs. 131–162. Springer, 2010.

[22] D. Golovin, B. Solnik, S. Moitra, G. Kochanski, J. Karro y D. Sculley. Google Vizier: a service for black-box optimization. En *Proceedings of the 23rd ACM SIGKDD Conference*, págs. 1487–1495, 2017.

[23] D. R. Jones, M. Schonlau y W. J. Welch. Efficient global optimization of expensive black-box functions. *Journal of Global Optimization*, 13:455–492, 1998.

[24] K. Kandasamy, G. Dasarathy, J. B. Oliva, J. Schneider y B. Póczos. Multi-fidelity Gaussian process bandit optimisation. Prepublicación, 2016 (versión de 2019). https://arxiv.org/abs/1603.06288

[25] J. Knowles. ParEGO: a hybrid algorithm with on-line landscape approximation for expensive multiobjective optimization problems. *IEEE Transactions on Evolutionary Computation*, 10(1):50–66, 2006.

[26] M. J. Kochenderfer y T. A. Wheeler. *Algorithms for Optimization*. MIT Press, 2019. Caps. 12 a 16. https://algorithmsbook.com/optimization/

[27] B. Lakshminarayanan, A. Pritzel y C. Blundell. Simple and scalable predictive uncertainty estimation using deep ensembles. Prepublicación, 2016 (versión de 2017). https://arxiv.org/abs/1612.01474

[28] L. Li, K. Jamieson, G. DeSalvo, A. Rostamizadeh y A. Talwalkar. Hyperband: a novel bandit-based approach to hyperparameter optimization. *Journal of Machine Learning Research*, 18, 2018. https://arxiv.org/abs/1603.06560

[29] M. Lindauer, K. Eggensperger, M. Feurer, A. Biedenkapp, D. Deng, C. Benjamins, T. Ruhopf, R. Sass y F. Hutter. SMAC3: a versatile Bayesian optimization package for hyperparameter optimization. Prepublicación, 2021. Licencia del código: BSD. https://arxiv.org/abs/2109.09831

[30] A. Makarova, H. Shen, V. Perrone, A. Klein, J. B. Faddoul, A. Krause, M. Seeger y C. Archambeau. Automatic termination for hyperparameter optimization. Prepublicación, 2021 (versión de 2022). https://arxiv.org/abs/2104.08166

[31] J. R. R. A. Martins y A. Ning. *Engineering Design Optimization*. Cambridge University Press, 2022. Caps. 9 y 10. https://mdobook.github.io/

[32] M. D. McKay, R. J. Beckman y W. J. Conover. A comparison of three methods for selecting values of input variables in the analysis of output from a computer code. *Technometrics*, 21(2):239–245, 1979. Reimpreso en *Technometrics*, 42(1):55–61, 2000.

[33] D. C. Montgomery. *Design and Analysis of Experiments*, 10.ª ed. Wiley. Caps. 1, 5, 6, 8 y 11.

[34] T. Akiba, S. Sano, T. Yanase, T. Ohta y M. Koyama. Optuna: a next-generation hyperparameter optimization framework. Prepublicación, 2019. Licencia del código: MIT. https://arxiv.org/abs/1907.10902

[35] The Pennsylvania State University. STAT 503: Design of Experiments. Curso en línea, lecciones 6 «The 2^k Factorial Design» y 11 «Response Surface Methods and Designs». https://online.stat.psu.edu/stat503/

[36] pyDOE3: Design of Experiments for Python. Licencia BSD-3. https://github.com/relf/pyDOE3

[37] M. Raissi, P. Perdikaris y G. E. Karniadakis. Physics-informed neural networks: a deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations. *Journal of Computational Physics*, 378:686–707, 2019.

[38] C. E. Rasmussen y C. K. I. Williams. *Gaussian Processes for Machine Learning*. MIT Press, 2006. Caps. 2, 4 y 5 y apéndice A. https://gaussianprocess.org/gpml/

[39] SciPy. `scipy.stats.qmc.LatinHypercube` y módulo `scipy.stats.qmc`. Licencia BSD. https://docs.scipy.org/doc/scipy/reference/stats.qmc.html

[40] scikit-learn. Gaussian Processes (guía de usuario, §1.7). Licencia BSD. https://scikit-learn.org/stable/modules/gaussian_process.html

[41] B. Shahriari, K. Swersky, Z. Wang, R. P. Adams y N. de Freitas. Taking the human out of the loop: a review of Bayesian optimization. *Proceedings of the IEEE*, 104(1):148–175, 2016.

[42] B. J. Shields, J. Stevens, J. Li, M. Parasram, F. Damani, J. I. Martinez Alvarado, J. M. Janey, R. P. Adams y A. G. Doyle. Bayesian reaction optimization as a tool for chemical synthesis. *Nature*, 590(7844):89–96, 2021.

[43] J. Snoek, H. Larochelle y R. P. Adams. Practical Bayesian optimization of machine learning algorithms. Prepublicación, 2012. https://arxiv.org/abs/1206.2944

[44] N. Srinivas, A. Krause, S. M. Kakade y M. Seeger. Gaussian process optimization in the bandit setting: no regret and experimental design. Prepublicación, 2009 (versión de 2010). https://arxiv.org/abs/0912.3995